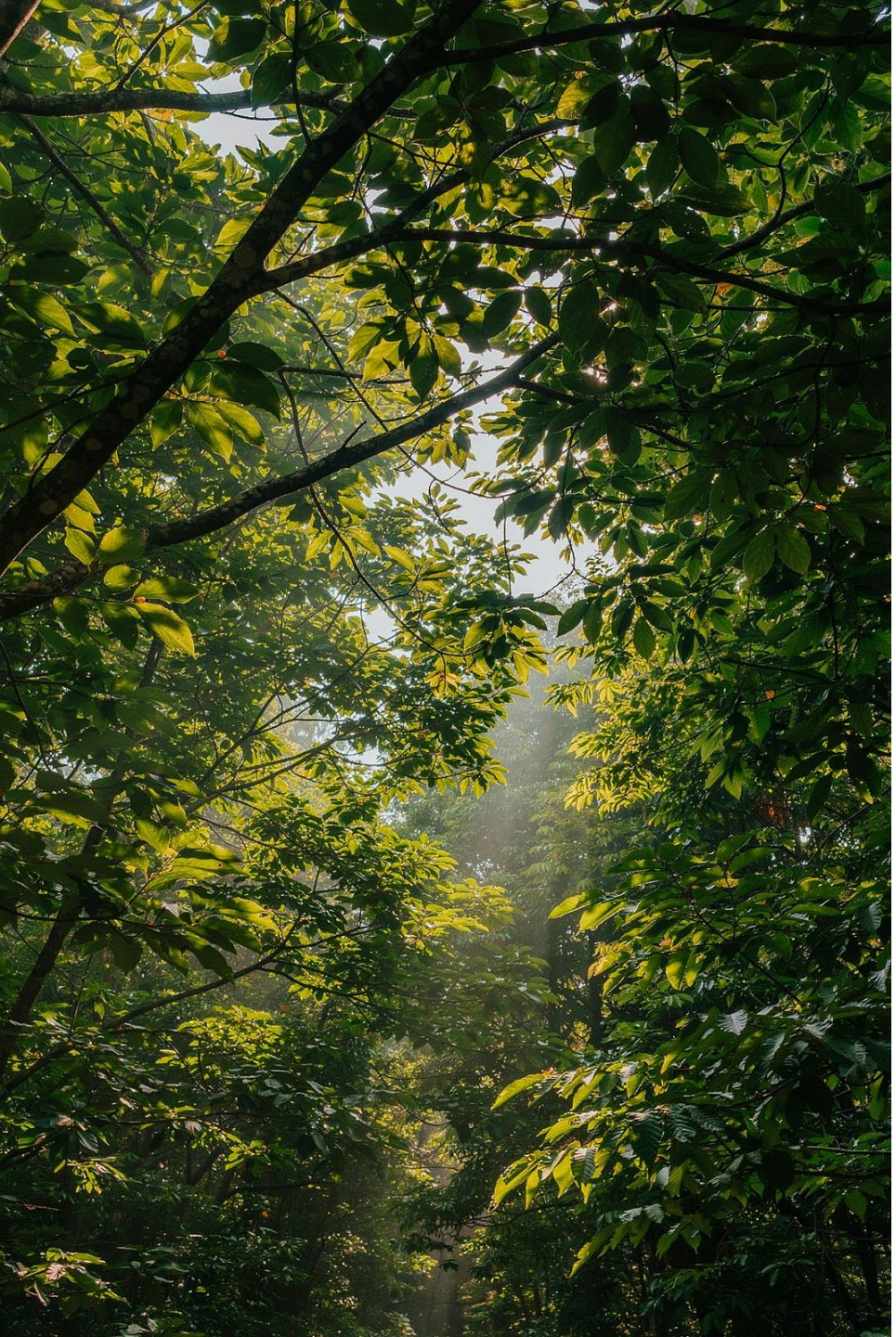


8.2 · Ensembles: Random Forest y Boosting

Los métodos de ensemble combinan múltiples modelos para obtener predicciones más robustas y precisas que cualquier modelo individual. En esta unidad exploraremos los dos grandes paradigmas: **Bagging** (con Random Forest como estandarte) y **Boosting** (con Gradient Boosting y XGBoost). También aprenderemos a interpretar estos modelos mediante la importancia de variables y los valores SHAP.

UNIDAD 8.2 APRENDIZAJE AUTOMÁTICO

¿Por qué combinar modelos?

Un único modelo predictivo siempre arrastra limitaciones: alto sesgo si es demasiado simple, o alta varianza si es demasiado complejo. La idea central de los **métodos de ensemble** es combinar múltiples modelos «débiles» para construir uno «fuerte», reduciendo simultáneamente ambos problemas.



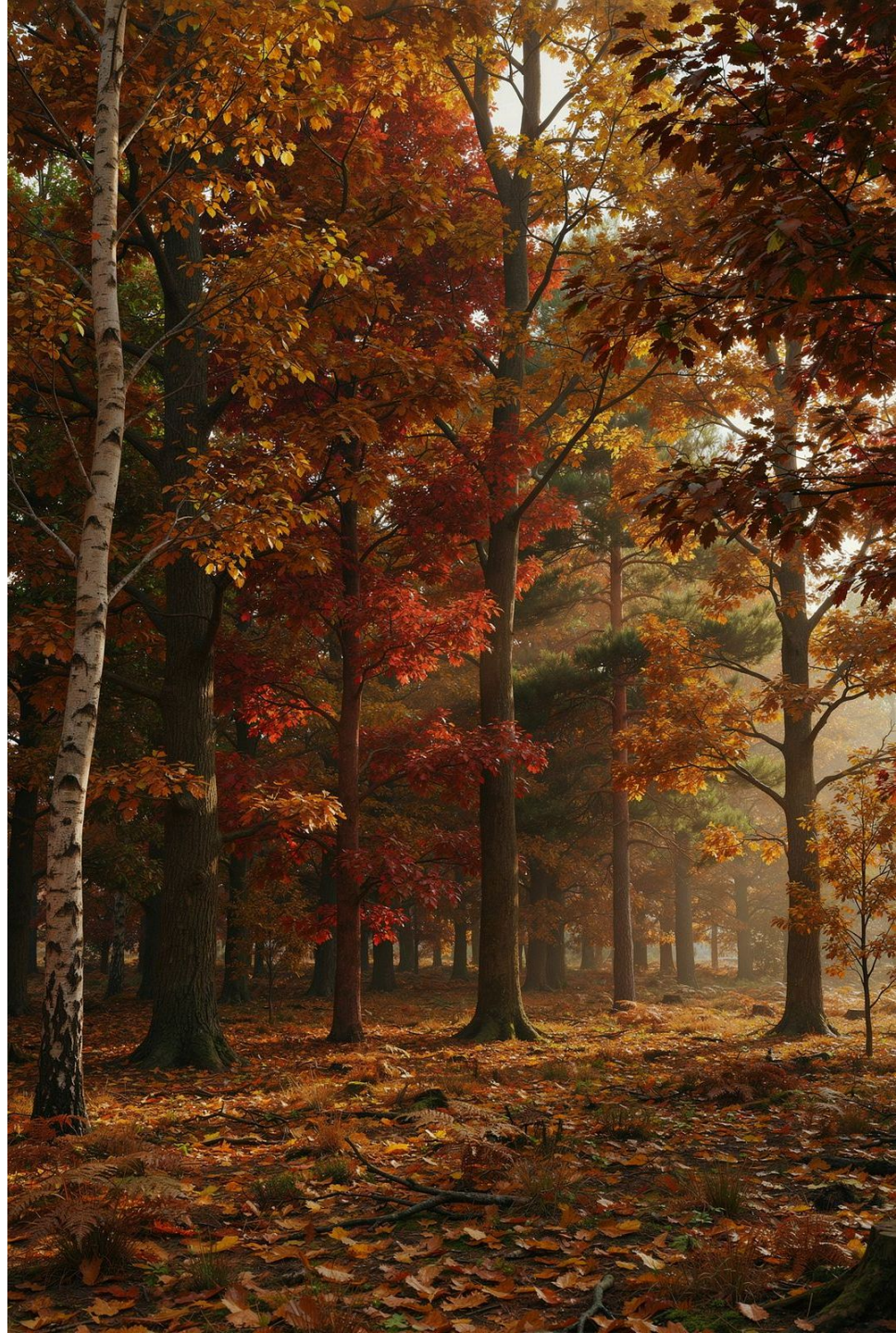
Bagging: Bootstrap Aggregating

El **Bagging** (Bootstrap Aggregating) entrena múltiples modelos base de forma independiente y en paralelo, cada uno sobre una muestra aleatoria con reemplazamiento del conjunto de entrenamiento. Las predicciones finales se obtienen por votación mayoritaria (clasificación) o promedio (regresión).



Random Forest

Random Forest extiende el Bagging añadiendo una segunda fuente de aleatoriedad: en cada división de cada árbol, solo se considera un subconjunto aleatorio de **m** características (típicamente **√p** para clasificación y **p/3** para regresión). Esto **decorrelaciona** los árboles, haciendo que sus errores sean más independientes y que el promedio sea más estable.



Importancia de variables en Random Forest

Random Forest ofrece dos medidas estándar para cuantificar la relevancia de cada predictor:

MDI — Mean Decrease in Impurity

Suma la reducción de impureza (Gini o varianza) provocada por cada variable a lo largo de todos los árboles y todos los nodos. Es rápida de calcular pero puede **favorecer variables de alta cardinalidad** y las codificadas como continuas.

Importancia por permutación

Aleatoriza los valores de una variable en el conjunto de validación y mide cuánto aumenta el error. Es más costosa pero **más fiable y agnóstica al modelo**. Disponible en scikit-learn como `permutation_importance`.

> Cuando las variables tienen escalas o cardinalidades muy distintas, prefiere la importancia por permutación para conclusiones más robustas.

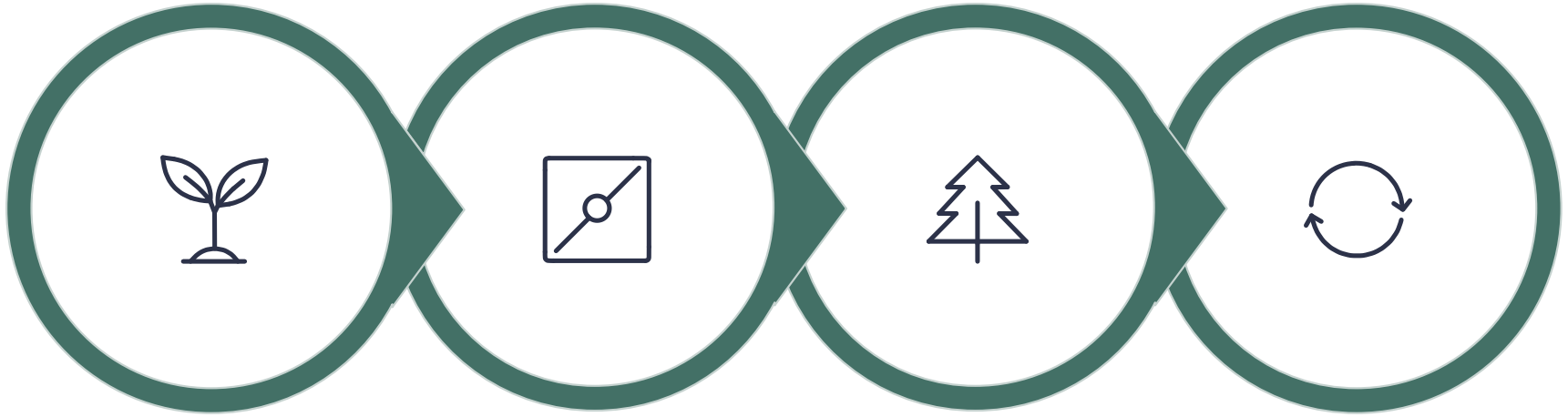


Boosting: aprendizaje secuencial

A diferencia del Bagging, el **Boosting** entrena los modelos base de forma **secuencial**: cada nuevo modelo se centra en los ejemplos que los anteriores clasificaron peor. El resultado final es una suma ponderada de modelos débiles que colectivamente forman un estimador potente con bajo sesgo.

Gradient Boosting: el algoritmo

Gradient Boosting enmarca el Boosting como un problema de **optimización funcional**: en cada iteración se ajusta un árbol a los **residuos negativos del gradiente** de la función de pérdida, avanzando en la dirección que más reduce el error.



Bagging vs. Boosting: comparativa directa

Bagging



1. Orden de Entrenamiento
En paralelo
independientes



2. Objetivo Principal
Reducir varianza



3. Modelos Base
Árboles profundos



4. Riesgo de Sobreajuste
Rara vez sobreajusta



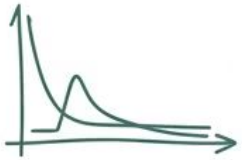
5. Velocidad
Rápido
paralelizable

6. Ejemplo
Random Forest

Boosting



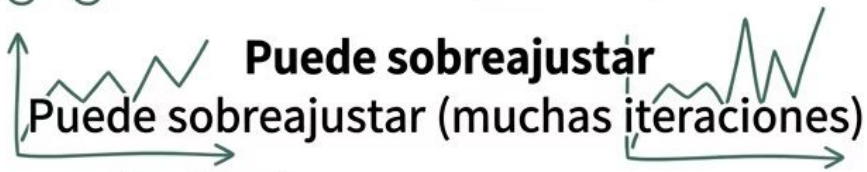
Secuencial
dependientes



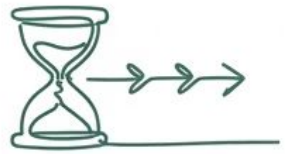
2. Objetivo Error
Reducir sesgo



3. Modelo Base
Tocones (shallow)



5. Velocidad
Más lento
secuencial



6. Ejemplo
XGBoost

XGBoost: Extreme Gradient Boosting

XGBoost es la implementación de Gradient Boosting más utilizada en competiciones y producción. Incorpora **regularización L1 y L2** en la función objetivo, lo que penaliza modelos complejos y reduce el sobreajuste. Añade además un término de complejidad basado en el número de hojas y sus pesos. Su implementación en C++ con soporte nativo para paralelismo lo hace significativamente más rápido que las implementaciones clásicas.



¿Cuándo usar Random Forest y cuándo XGBoost?



Elige Random Forest cuando...

- Necesitas un modelo robusto con mínimo ajuste de hiperparámetros
- El tiempo de entrenamiento es limitado (paralelizable)
- El conjunto de datos tiene muchas variables irrelevantes
- Priorizas estabilidad y reproducibilidad



Elige XGBoost cuando...

- Buscas el máximo rendimiento predictivo
- Dispones de tiempo para ajustar hiperparámetros
- El problema es tabular y el error de sesgo es dominante
- Participas en competiciones de machine learning

S08 · EN EL REPOSITORIO DEL CURSO

Ahora, a practicar

Lo que viene a continuación está en tu repositorio; no hace falta buscar ejercicios fuera.

CUADERNO

S08_02_random_forest — 40 min

Random Forest: bagging, OOB e importancia por permutación

sprints/S08/notebooks/S08_02_random_forest.ipynb

CUADERNO

S08_03_boosting_xgboost_lightgbm — 45 min

Gradient Boosting, XGBoost y LightGBM

sprints/S08/notebooks/S08_03_boosting_xgboost_lightgbm.ipynb

CUADERNO

S08_06_leaderboard_plantilla — 25 min

Primeros modelos en el *leaderboard* (*baseline*, logística, árbol y bosque; pasos 1–2)

sprints/S08/notebooks/S08_06_leaderboard_plantilla.ipynb